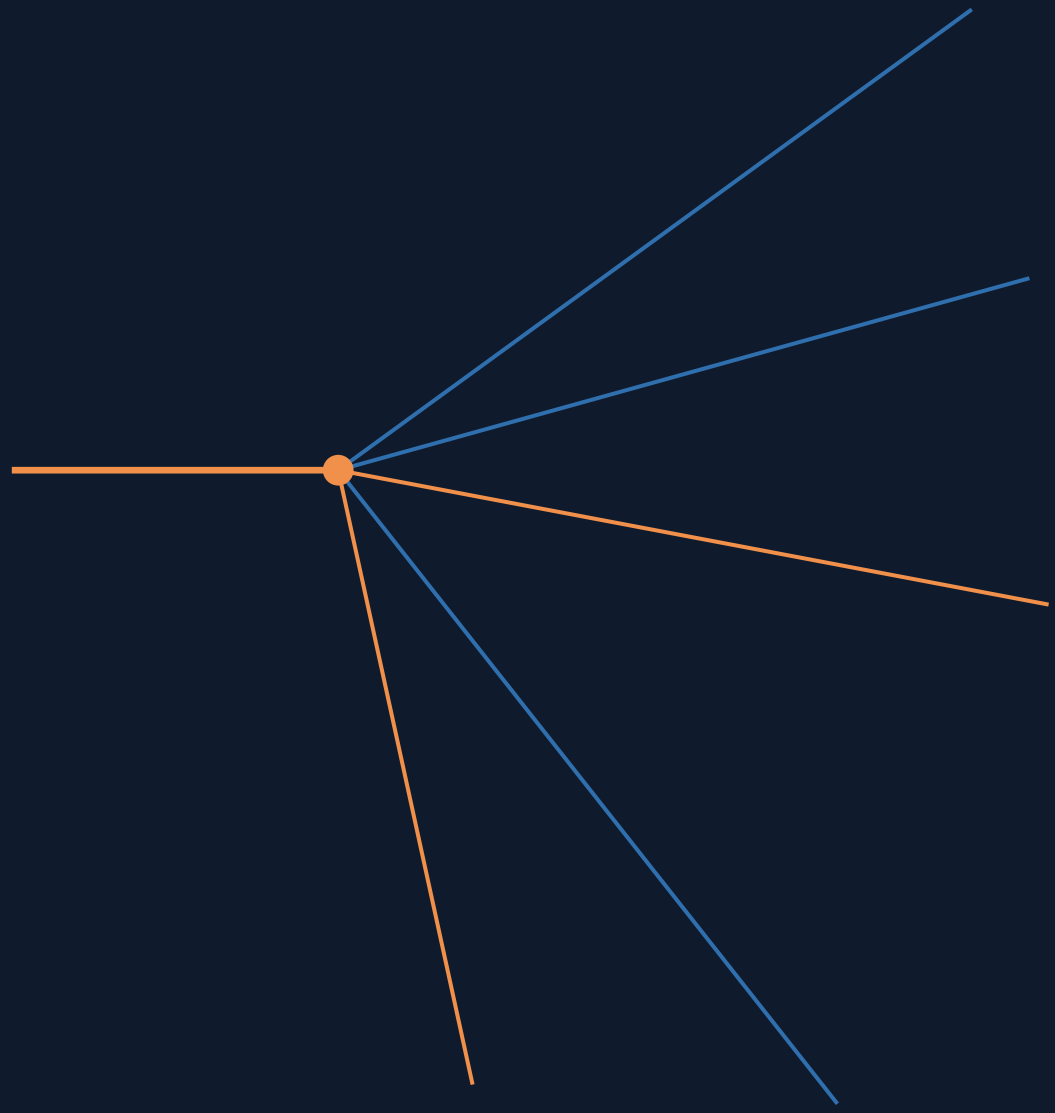
PARÇACIK DETEKTÖRLERİ TASARIM VE SİMULASYONLARI

Geant4 ile Kodlama

İlk simulasyon yazımı, Bragg Tepesi ve Süreç Örnekleri Uygulamaları



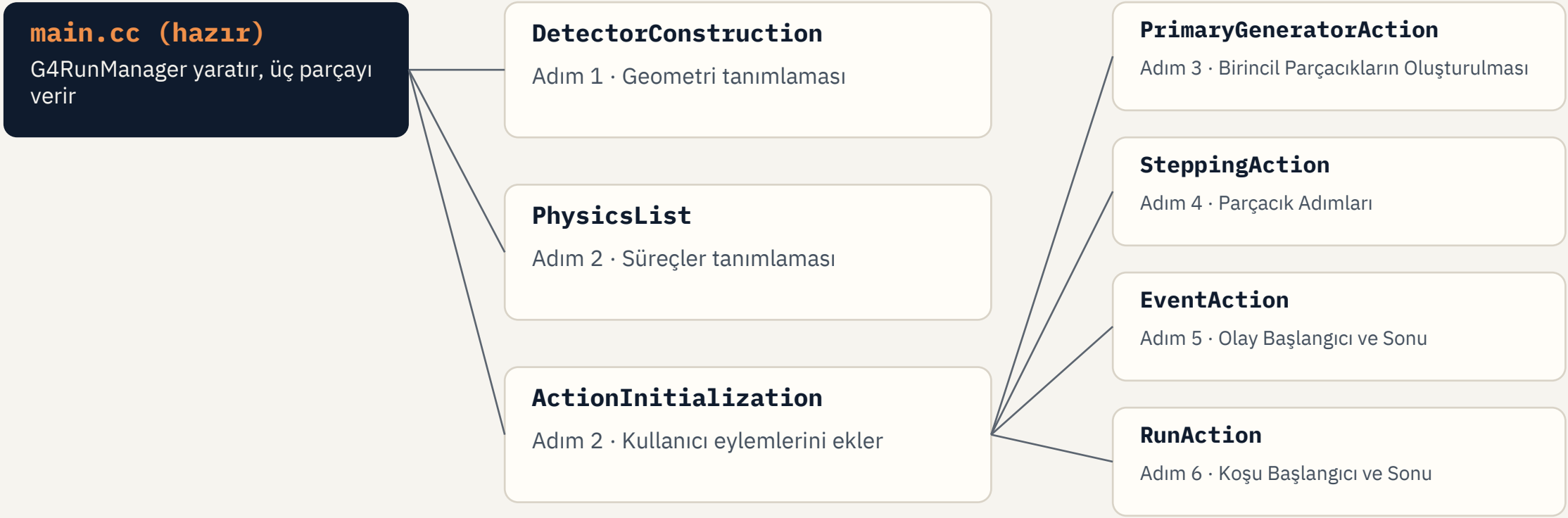
Kaan Yüksel OYULMAZ · Ankara Üniversitesi Hızlandırıcı Teknolojileri Enstitüsü · 2026

GEANT4 İLE KODLAMA

Bu derste ne yapacağız?

00	**Sınıflar nasıl çalışır?**	Geant4 çağırır, biz doldururuz; koşu / olay / adım
01	**Geometri**	DetectorConstruction: boşluk dolu dünya + 300 µm Si levha
02	**Fizik listesi**	PhysicsList: hangi süreçler simüle edilecek
03	**Parçacık kaynağı**	PrimaryGeneratorAction: G4ParticleGun, μ^- 1 GeV
04	**SteppingAction**	Levhada mı? Levhada ise bırakılan enerjiyi olaya ekle
05	**EventAction**	Olay başına toplam → Olay bitince histogramı doldur
06	**RunAction**	Histogram tanımı; dosyayı aç / yaz / kapat
07	**Sonuç**	ROOT ile analiz: Landau dağılımı
08	**Bragg Tepesi**	Aynı kod üzerine: Su, 20 cm genişlik, proton kaynağı; Bragg Tepesi histogramı
09	**Süreçlerin payı**	Rastgele enerji; ikincillerin yaratıcı süreci; iyonlaşma / frenleme / çift

Sınıflar nasıl çalışır?



Ana fikir: Biz blokları doldururuz, Geant4 çağırır. Her adım için bir dosyanın gövdesini yazacağız.

Ne zaman çağrılır? Koşu, olay, adım

Koşu (run)

/run/beamOn 10000

RunAction

`BeginOfRunAction` · `EndOfRunAction`

1 kez: dosyayı aç; sonda yaz ve kapat

Olay (event)

1 müon + ürettiği o dünyadaki bütün etkileşimleri

EventAction

`BeginOfEventAction` · `EndOfEventAction`

10 000 kez: fEdep = 0; sonda FillH1(0, fEdep)

Adım (step)

İki etkileşme / sınır arası

SteppingAction

`UserSteppingAction`

Yüz binlerce kez: edep'i olaya ekle

> **Veri toplama kuralı:** Adımda ölç, olayda topla, koşuda yaz.

Boş Geant4 iskeleti: G4Ders

Dosya ağacı (turuncu: derste doldurulacak)

```
G4Ders/
  CMakeLists.txt
  main.cc                 # hazır
  macros/mip_si.mac       # hazır
  include/                # 7 başlık, hazır
  src/
    ActionInitialization.cc    # hazır
    DetectorConstruction.cc    # adım 1
    PhysicsList.cc             # adım 2
    PrimaryGeneratorAction.cc  # adım 3
    SteppingAction.cc          # adım 4
    EventAction.cc             # adım 5
    RunAction.cc               # adım 6
```

derleme ve koşu (WSL)

```
source /opt/geant4/bin/geant4.sh
cd G4Ders
mkdir build
cd build
cmake ../.
make -j4
```

- **Boş gövde:** Her fonksiyon { } şeklinde
- **Başlıklar hazır:** Üye değişkenler ve fonksiyon imzaları yazılı ve biz gövdeleri dolduruyoruz.
- **Boşken:** Hatasız derlenir; koşunca dünya olmadığı için çöker (Segmentation fault). İlk iş: Construct() fonksiyonun dolması.
- **Her değişiklikten sonra:** make –j4; ./ders mip_si.mac

Üç katmanlı geometri: Solid, Mantıksal, Fiziksel

1 · Solid — G4Box

Şekil ve boyut. Yarı boyutlar verilir: 300 µm kalınlık için dz = 150 µm.

```
new G4Box("Silicon", 5*cm, 5*cm, 150*um)
```

2 · Mantıksal hacim — G4LogicalVolume

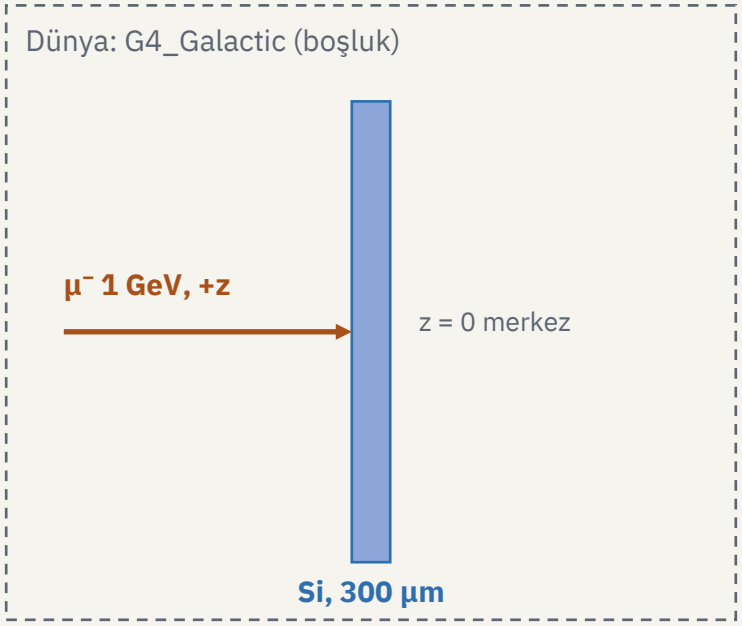
Şekil + malzeme.

```
new G4LogicalVolume(solidSi, silicon, "Silicon")
```

3 · Fiziksel hacim — G4PVPlacement

Mantıksal hacmi ana hacmin içine, bir konuma koyar.

```
new G4PVPlacement(nullptr, G4ThreeVector(), logicSi, "Silicon", logicWorld, false, 0)
```



Malzeme: NIST adıyla: `G4_Si`, `G4_Galactic`
Dünya en dış hacimdir; Construct onu döndürür.

DetectorConstruction::Construct()

```
G4VPhysicalVolume* DetectorConstruction::Construct()
{
  // 1) malzemeler (NIST adıyla)
  G4NistManager* nist = G4NistManager::Instance();
  G4Material* vacuum  = nist->FindOrBuildMaterial("G4_Galactic");
  G4Material* silicon = nist->FindOrBuildMaterial("G4_Si");

  // 2) Dünya: boşluk dolu kutu (yarı boyutlar!)
  G4Box* solidWorld = new G4Box("World", 10*cm, 10*cm, 10*cm);
  G4LogicalVolume* logicWorld = new G4LogicalVolume(solidWorld, vacuum, "World");
  G4VPhysicalVolume* physWorld = new G4PVPlacement(nullptr, G4ThreeVector(),
                                  logicWorld, "World", nullptr, false, 0);

  // 3) Levha: merkezde, 300 um silisyum (yarı kalınlık 150 um)
  G4Box* solidSi = new G4Box("Silicon", 5*cm, 5*cm, 150*um);
  G4LogicalVolume* logicSi = new G4LogicalVolume(solidSi, silicon, "Silicon");
  new G4PVPlacement(nullptr, G4ThreeVector(), logicSi, "Silicon",
                    logicWorld, false, 0);

  return physWorld;   // 4) dünya döndürülür
}
```

- **Birimler:** Her uzunluk birimle çarpılır: 150*um, 5*cm. Birimsiz sayı mm sanılır!
- **Yarı boyutlar:** G4Box tam boy değil, yarı boy ister: 300 µm için 150*um.
- **G4ThreeVector():** (0,0,0) konumu; levha ve dünya aynı merkezde.
- **Adlar:** "Silicon" fiziksel hacmin adı ve adım 4'te "levhada mıyım?" sorusu bu adla sorulacak.

Kontrol noktası: Derle, ./ders mip_si.mac → Artık çökmez; "geometri kuruldu" ama parçacık kaynağı yok bu yüzden beamOn hata verir çünkü fizik listesi daha tanımlanmadı.

Nereye bakmalı: Şekiller ve malzemeler

Şekiller (solid)

Sınıf	Şekil ve parametreler
G4Box	kutu: yarı boyutlar dx, dy, dz
G4Tubs	silindir / boru: iç-dış yarıçap, yarı boy, açı
G4Cons	koni: iki uçta yarıçaplar, yarı boy
G4Sphere	küre / kabuk: yarıçaplar, açılar
G4Trd	yamuk prizma: iki uçta dx, dy; dz
G4UnionSolid	Boole: birleşim, fark, kesişim

```
new G4Tubs("Pipe", 0, 2*cm, 10*cm, 0, 360*deg)
```

Belge: Application Developer Guide → Detector → Geometry → Solids
geant4.web.cern.ch/documentation/…/Detector/Geometry/geomSolids.html

Malzemeler (NIST adları)

Ad	Malzeme
G4_Si, G4_Al, G4_Pb	elementler: G4_ + sembol
G4_WATER, G4_AIR	bileşik / karışım
G4_PLASTIC_SC_VINYLTOLUENE	plastik sintilatör
G4_SODIUM_IODIDE	NaI kristali
G4_Galactic	boşluk (dünya için)
G4_lAr, G4_Ge	sıvı argon, germanyum

```
/material/nist/listMaterials      # tüm liste terminale
```

Belge: Application Developer Guide → Appendix → NIST Material Names
geant4.web.cern.ch/documentation/…/Appendix/materialNames.html

Alışkanlık: Yeni bir şekil ya da malzeme gerektiğinde önce bu iki sayfaya bakın

PhysicsList: Hangi süreçler simüle edilecek?

```
// PhysicsList.cc: hazır yapı taşları kaydedilir
PhysicsList::PhysicsList()
{
  RegisterPhysics(new G4EmStandardPhysics_option4()); // elektromanyetik
  RegisterPhysics(new G4DecayPhysics());              // bozunmalar
  RegisterPhysics(new G4HadronElasticPhysics());      // hadron saçılma
  RegisterPhysics(new G4HadronPhysicsFTFP_BERT());    // nükleer tepkime
  RegisterPhysics(new G4IonPhysics());                // alfa, iyonlar

  SetDefaultCutValue(0.1 * mm);                       // üretim kesimi
}
```

- **Ayrı sınıf:** PhysicsList : G4VModularPhysicsList. Kurucuda RegisterPhysics.
- **option4:** En hassas EM fizik. İnce katmanda doğru dE/dx için şart; bugünkü ölçüm buna dayanıyor.
- **Bugün kullanılan:** yalnızca EM kısmı (müon iyonlaşması). Diğerleri ileriki dersler için.
- **Kesim 0,1 mm:** silisyumda ≈ 80 keV: bunun altındaki δ-elektronları ayrı iz olmaz, enerjileri yerinde bırakılır.
- **main.cc'de (hazır):** runManager->SetUserInitialization(new PhysicsList());

Parçacık	Süreç adı	Fizik (geçen ders)
μ, p, α	`muIoni, hIoni, ionIoni`	iyonlaşma: Bethe–Bloch
e^-, e^+	`eIoni, eBrem`	iyonlaşma, frenleme ışıması
yüklü hepsi	`msc`	çoklu Coulomb saçılması

Kontrol noktası: Derle, çalıştır → run initialize geometri + fizik kurar; beamOn artık çalışır ama parçacık üretilmez (kaynak yok).

PrimaryGeneratorAction: G4ParticleGun

```
PrimaryGeneratorAction::PrimaryGeneratorAction()
{
  fGun = new G4ParticleGun(1);   // olay başına 1 parçacık

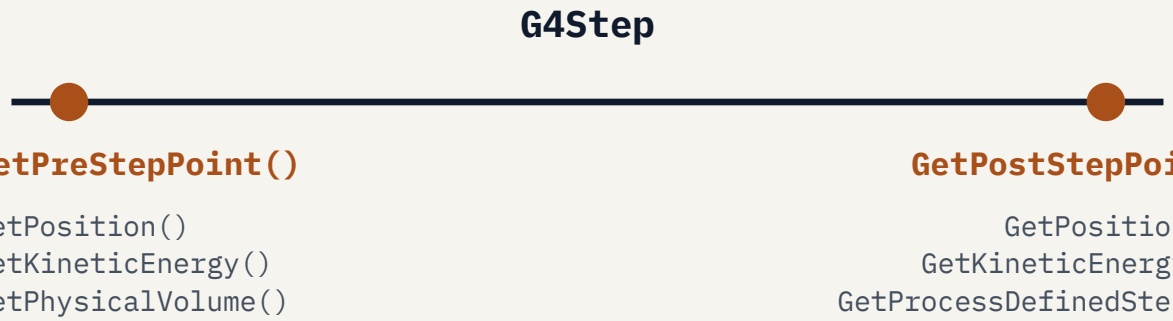
  G4ParticleDefinition* particle =
      G4ParticleTable::GetParticleTable()->FindParticle("mu-");
  fGun->SetParticleDefinition(particle);
  fGun->SetParticleEnergy(1 * GeV);
  fGun->SetParticleMomentumDirection(G4ThreeVector(0., 0., 1.));  // +z
  fGun->SetParticlePosition(G4ThreeVector(0., 0., -1 * cm));  // önde
}

void PrimaryGeneratorAction::GeneratePrimaries(G4Event* event)
{
  fGun->GeneratePrimaryVertex(event);   // parçacığı olaya ekle
}
```

- **G4ParticleGun:** En basit kaynak tanımı, tek parçacık, sabit konum ve yön.
- **FindParticle("mu-"):** Parçacık tablosundan isimle: e-, proton, alpha, gamma...
- **Konum:** z = −1 cm; Dünya boşluk, müon levhaya tam 1 GeV ile gelir.
- **Hazır komutlar:** /gun/particle, /gun/energy; makrodan değiştirilir.
- **GeneratePrimaryVertex:** Unutulursa olay boş geçer ve hata vermez.

Kontrol noktası: Derle, ./ders mip_si.mac → 10 000 müon levhadan geçer; terminalde koşu biter. Henüz ölçüm yok.

G4Step'in içinde ne var?



Adımın kendisi: `GetTotalEnergyDeposit()` bu adımda bırakılan enerji — bugünkü tek ölçüm.
Adımı atan iz: `GetTrack()` → G4Track: `GetVolume()->GetName()`, `GetTrackID()`, `GetKineticEnergy()`, `GetDefinition()` …

- **Bugün iki soru:** (1) adım levhada mı? İzin bulunduğu hacmin adı: GetTrack()->GetVolume()->GetName() (2) ne kadar enerji bırakıldı? GetTotalEnergyDeposit() kullanılır.
- **Enerji bırakma:** Noktanın değil, adımın özelliği.
- **Müon 300 µm Si'de:** ≈ 6 adım, her adımda 10–30 keV; toplamı olay başına ≈ 100 keV.
- **Sonraki örneklerde:** İkinciller, derinlik hesabı ve süreçler.

SteppingAction::UserSteppingAction()

```
void SteppingAction::UserSteppingAction(const G4Step* step)
{
  // 1) adımın başladığı hacmin adı: adım -> iz -> hacim -> ad
  G4String volumeName = step->GetTrack()->GetVolume()->GetName();
  if (volumeName != "Silicon") return;   // levha değilse ilgilenme

  // 2) bu adımda bırakılan enerji -> olaya ekle
  G4double edep = step->GetTotalEnergyDeposit();
  fEvent->AddEdep(edep);
}
```

```
// EventAction.hh (hazır):
void AddEdep(G4double e) { fEdep += e; }   // SteppingAction her adımda çağırır
```

- **Her adımda çağrılır:** Yüz binlerce kez; sadece ölçer, EventAction'a iletir (B1 örneğindeki gibi).
- **GetTrack():** Adımı atan iz (G4Track). Oradan hacim, enerji, iz numarası, parçacık türü: hepsine bu kapıdan ulaşılır.
- **GetVolume()->GetName():** adımın başladığı fiziksel hacmin adı; adım 1'de "Silicon" demiştik. Dünyadaysa "World" gelir, çıkılır.
- **Histogram yok:** Adımda ölç, olayda topla, koşurmada yaz.

Kontrol noktası: Derle, çalıştır → değişen bir şey görünmez: enerji toplanıyor ama henüz hiçbir yere yazılmıyor.

EventAction: Olay Toplam Enerji Sayımı

```
void EventAction::BeginOfEventAction(const G4Event*)
{
  fEdep = 0.;   // yeni olay: toplamı sıfırla
}

void EventAction::EndOfEventAction(const G4Event*)
{
  G4AnalysisManager::Instance()->FillH1(0, fEdep);   // histograma yaz
}
```

```
// EventAction.hh (hazır):
void AddEdep(G4double e) { fEdep += e; }
private:
  G4double fEdep = 0.;   // bu olayda levhada bırakılan toplam enerji
```

- **Başta sıfırla, sonda yaz:** Olay başına tek değişken: fEdep.
- **FillH1(0, fEdep):** 0 = histogram numarası (adım 6'da tanımlanacak); değer iç birimde (MeV).
- **G4AnalysisManager:** Geant4'ün analiz için kayıt aracı, tek nesne, Instance() ile erişilir.
- **Olay başına bir giriş:** 10 000 olay → 10 000 giriş → dağılım.

Kontrol noktası: Derle, çalıştır → henüz dosya yok: histogram tanımlanmadığı için dosya açılmadı.

RunAction: histogram, dosya aç / yaz / kapat

```
RunAction::RunAction()
{
  G4AnalysisManager* am = G4AnalysisManager::Instance();
  am->SetDefaultFileType("root");
  am->SetFileName("ders");
  am->CreateH1("edep", "Olay basina birakilan enerji",
               200, 0., 500.*keV, "keV");
}

void RunAction::BeginOfRunAction(const G4Run*)
{
  G4AnalysisManager::Instance()->OpenFile();   // dosyayı aç
}

void RunAction::EndOfRunAction(const G4Run* run)
{
  G4AnalysisManager* am = G4AnalysisManager::Instance();
  am->Write();       // histogramı yaz
  am->CloseFile();   // dosyayı kapat
  G4cout << "Olay sayisi: " << run->GetNumberOfEvent() << G4endl;
}
```

- **Histogram id:** tanımlama sırası; ilk CreateH1 → 0. EventAction FillH1(0, ...) buraya yazar.
- **Birimler:** sınırlar iç birimde: 500.*keV. "keV" yalnızca ekseni etiketler; FillH1'e de iç birim (MeV) verilir.
- **OpenFile / Write / CloseFile:** koşu başı / sonu. CloseFile unutulursa dosya boş.
- **Dosya adı:** makrodan /analysis/setFileName mip_si → mip_si.root.

Kontrol noktası: derle, ./ders mip_si.mac → "Olay sayisi: 10000" ve mip_si.root. Ölçüm tamam.

Ne yaptık? Kodun akışı

olay başlar

PrimaryGeneratorAction

kaynak · her olayda 1 kez

μ^- 1 GeV, z = −1 cm, +z yönünde ateşlenir

adım adım

DetectorConstruction

geometri · 1 kez kurulur

boşluk dolu dünya; 300 µm Si levha, z = 0

SteppingAction

her adımda (× ~6 / olay)

levhada mı? edep → fEvent->AddEdep(edep)

olay sonu: fEdep

EventAction

olay başı / sonu (× 10 000)

fEdep = 0 … FillH1(0, fEdep)

koşu sonu

RunAction

koşu başı / sonu (× 1)

CreateH1; OpenFile … Write, CloseFile

dosya

ROOT · draw_landau.C

koşu bittikten sonra

mip_si.root → edep histogramı → Landau

Kural: adımda ölç (SteppingAction), olayda topla (EventAction), koşuda yaz (RunAction).

Makro ile koşturmak

makro

```
# mip_si.mac
/run/initialize           # geometri + fizik kurulur

/gun/particle mu-
/gun/energy 1 GeV

/analysis/setFileName mip_si
/run/printProgress 1000
/run/beamOn 10000         # koşu: 10 000 olay
```

derleme ve koşu (WSL)

```
source /opt/geant4/bin/geant4.sh
cmake -S ~/G4Ders -B ~/G4Ders-build
cmake --build ~/G4Ders-build -j12

cd ~/G4Ders-build
./ders mip_si.mac
root -l draw_landau.C
```

- **Sıra:** /gun/ komutları /run/initialize'dan sonra.
- **Başka parçacık:** /gun/particle proton, /gun/energy 100 MeV — kod değişmez.
- **Argümansız ./ders:** Görsel arayüz, nesneler ve izler görülür (VcXsrv açık olmalı).

ROOT ile analiz: draw_landau.C

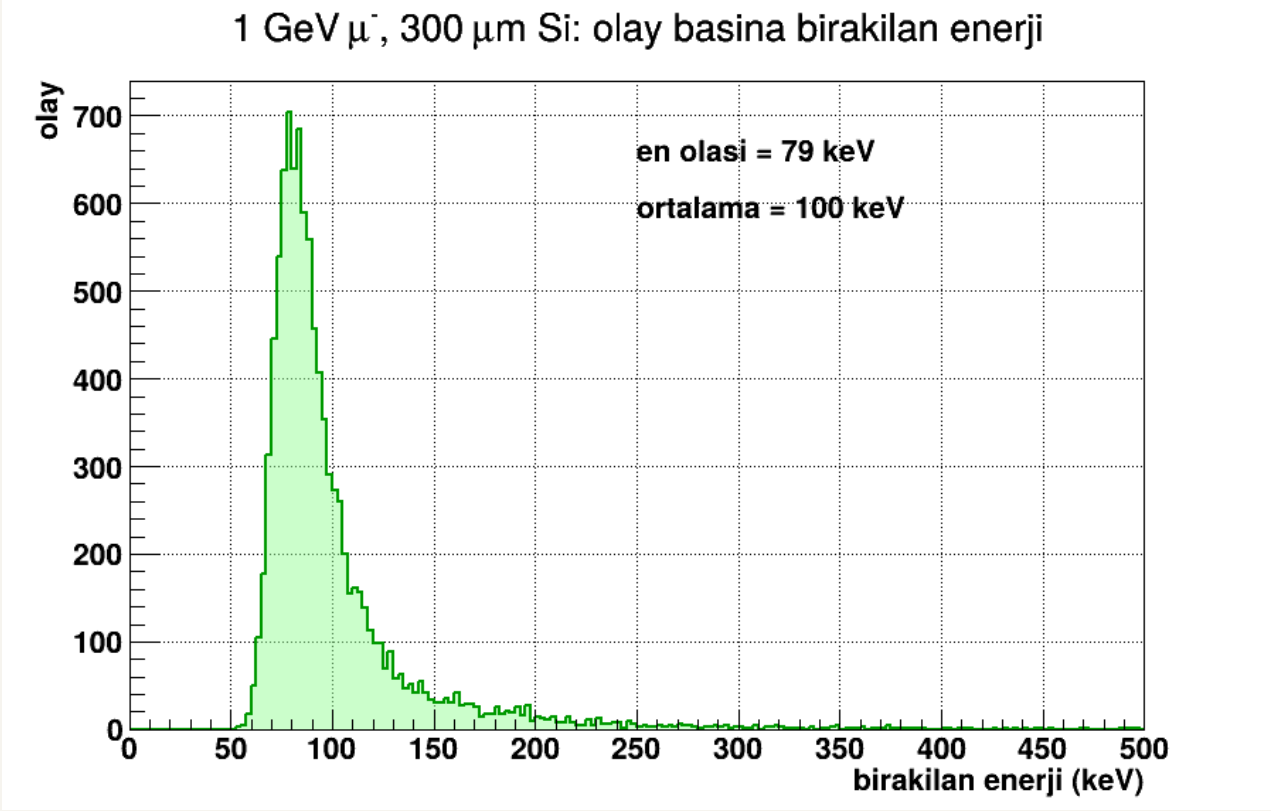
```
// draw_landau.C — mip_si.root içindeki edep histogramını çizer
void draw_landau()
{
  TFile* f = TFile::Open("mip_si.root");
  TH1* h = (TH1*)f->Get("edep");

  TCanvas* c = new TCanvas("c", "", 900, 600);
  h->SetTitle("1 GeV mu-, 300 um Si;birakilan enerji (keV);olay");
  h->SetLineColor(kGreen + 2);
  h->Draw("hist");

  double mpv  = h->GetBinCenter(h->GetMaximumBin());   // en olasi deger
  double mean = h->GetMean();                          // ortalama
  printf("en olasi = %.0f keV, ortalama = %.0f keV\n", mpv, mean);
  c->SaveAs("landau.png");
}
```

- **TFile::Open / Get:** Dosyayı aç, histogramı adıyla al ("edep").
- **Draw("hist"):** Çizgi olarak çiz.
- **En olası vs ortalama:** Tepenin yeri ve aritmetik ortalama; Landau'da farklı.
- **Elle de olur:** root -l mip_si.root → edep->Draw()

Sonuç: Landau dağılımı



- **Neden asimetrik?** ince levhada az çarpışma; seyrek büyük aktarım (δ-ışını) sağa kuyruk yapar. En olası < ortalama.
- **Deneyin:** /gun/energy 10 MeV ya da /gun/particle proton: dağılım nasıl değişir?

Bragg Eğrisi Örneği

1 **DetectorConstruction**

G4_Si → G4_WATER; 300 µm → 20 cm, merkezde. Kaynak 10 cm geriye: z = −11 cm

2 **SteppingAction**

adımın derinliği z + 10 cm; fEvent->AddDepth(z, edep)

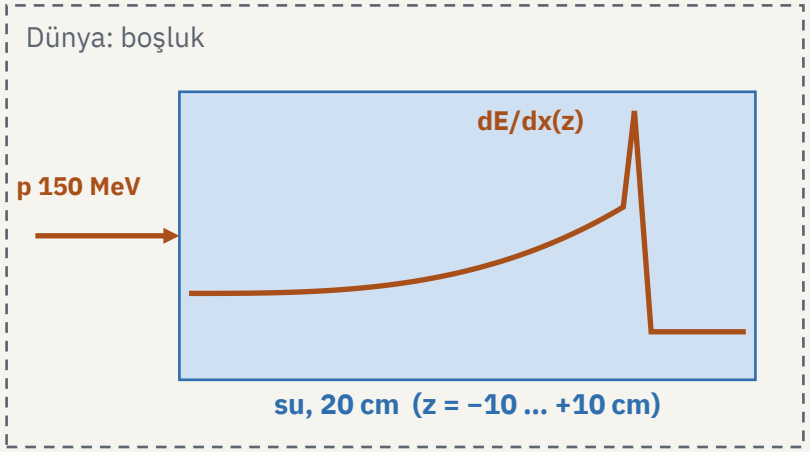
3 **RunAction**

ikinci histogram: CreateH1("depth", ..., 0., 20.*cm, "cm") → id 1

4 **EventAction**

AddDepth(z, edep): FillH1(1, z, edep); ağırlık = enerji

Değişmeyen: PhysicsList, main.cc. PrimaryGeneratorAction'da yalnızca konum. Makroda: /gun/particle proton, /gun/energy 150 MeV.



Derinlik histogramı: her adımda x = z yazılır, **ağırlık = edep**. Kutu içeriği = o derinlikte bırakılan enerji.
Kutu 1 mm, 1000 proton: 1000'e bölünce y ekseni MeV/mm, yani dE/dx.
Beklenti (geçen ders): 150 MeV protonun suda menzili ≈ 15,8 cm; pik oradadır.

Değişiklik 1: DetectorConstruction, Su bloğu

```
G4VPhysicalVolume* DetectorConstruction::Construct()
{
  G4NistManager* nist = G4NistManager::Instance();
  G4Material* vacuum = nist->FindOrBuildMaterial("G4_Galactic");
  G4Material* water  = nist->FindOrBuildMaterial("G4_WATER");
  G4Box* solidWorld = new G4Box("World", 25*cm, 25*cm, 25*cm);  // büyüdü
  G4LogicalVolume* logicWorld = new G4LogicalVolume(solidWorld, vacuum, "World");
  G4VPhysicalVolume* physWorld = new G4PVPlacement(nullptr, G4ThreeVector(),
                                   logicWorld, "World", nullptr, false, 0);
  // su bloğu: merkezde, 20 cm uzun (yarı 10 cm): z = -10 ... +10 cm
  G4Box* solidWater = new G4Box("Water", 5*cm, 5*cm, 10*cm);
  G4LogicalVolume* logicWater = new G4LogicalVolume(solidWater, water, "Water");
  new G4PVPlacement(nullptr, G4ThreeVector(), logicWater, "Water",
                    logicWorld, false, 0);
  // renk: mavi, yarı saydam, dolu gösterim  (#include "G4VisAttributes.hh")
  G4VisAttributes* blue = new G4VisAttributes(G4Colour(0.2, 0.5, 1.0, 0.4));
  blue->SetForceSolid(true);
  logicWater->SetVisAttributes(blue);
  return physWorld;
}
```

- **Turuncu satırlar:** Değişenler. Gerisi aynı.
- **G4_WATER:** NIST adı; yoğunluk 1 g/cm³.
- **Yarı uzunluk 10 cm:** Blok 20 cm. Dünya 25 cm yarı boyut ki blok dünyanın içinde kalsın!
- **Merkezde:** G4ThreeVector(); blok z = −10 … +10 cm.
- **Kaynak geriye:** PrimaryGeneratorAction'da SetParticlePosition z = −11 cm; yoksa proton suyun içinde doğar.
- **Renk:** G4VisAttributes + G4Colour(r, g, b, saydamlık), her biri 0–1; SetForceSolid dolu çizer. Yalnızca görünüm.

```
// PrimaryGeneratorAction.cc, kurucu
fGun->SetParticlePosition(
    G4ThreeVector(0., 0., -11*cm));
```

Kontrol noktası: Derle, ./ders bragg.mac → 1000 proton suda durur ve edep histogramı taşar (150 MeV), önemli değil. Derinlik henüz yok.

Değişiklik 2: SteppingAction'da Derinlik Hesabı

```
void SteppingAction::UserSteppingAction(const G4Step* step)
{
  G4String volumeName = step->GetTrack()->GetVolume()->GetName();
  if (volumeName != "Water") return;   // ad değişti

  G4double edep = step->GetTotalEnergyDeposit();
  fEvent->AddEdep(edep);

  // orta nokta; su z = -10 cm'de başlar -> derinlik = z + 10 cm
  G4double z1 = step->GetPreStepPoint()->GetPosition().z();
  G4double z2 = step->GetPostStepPoint()->GetPosition().z();
  G4double z = 0.5 * (z1 + z2) + 10*cm;
  fEvent->AddDepth(z, edep);
}
```

```
// EventAction.hh: yeni fonksiyonun imzası (public kısmına)
  void AddDepth(G4double z, G4double edep);
```

- **Ad değişti:** "Silicon" yerine "Water"; adla karşılaştırdığımız için tek kelime.
- **GetPosition():** Adım noktasının z-ekseni boyunca derinlik yönü.
- **Orta nokta:** Suda adım birkaç mm
- **+ 10 cm:** blok −10 cm'de başlar. G4SystemOfUnits.hh kütüphanesi gerekir.
- **AddDepth(z, edep):** Ölçüm önceki örnek gibi yine SteppingAction'da, yazma EventAction'da.
- **Başlık dosyası:** İmza fonsiyonu eklenmeden derlenmez. Aksi taktirde "AddDepth is not a member« hatası.

Kontrol noktası: Henüz derlenmez: AddDepth'in gövdesi yok. Değişiklik 3 ve 4'ten sonra derleyin.

Değişiklik 3 ve 4: Histogramı tanımlama ve doldurma

```
// RunAction.cc, kurucu: ikinci histogram (id = 1)
  am->CreateH1("edep",  "Olay basina birakilan enerji",
               200, 0., 500.*keV, "keV");
  am->CreateH1("depth", "Derinlige gore birakilan enerji",
               200, 0., 20.*cm, "cm");   // kutu = 1 mm
```

```
// EventAction.cc: her adımda SteppingAction çağırır
void EventAction::AddDepth(G4double z, G4double edep)
{
  G4AnalysisManager::Instance()->FillH1(1, z, edep);   // ağırlık = edep
}

// Begin/EndOfEventAction aynı: fEdep = 0 ... FillH1(0, fEdep)
```

- **id = 1:** İkinci tanımlanan histogram. FillH1(1, ...) ile eşleşmeli.
- **FillH1(id, x, ağırlık):** x = z (iç birim mm, eksen cm); ağırlık = bırakılan enerji.
- **Ağırlıksız olsa:** Adım sayısını sayardı; bize enerji lazım.
- **Olay sonunda değil:** Derinlik her adımda yazılır; toplam enerji ise olay sonunda.

Kontrol noktası: Derle, ./ders bragg.mac → Olay sayisi: 1000 olarak belirle, bragg.root içinde edep ve depth ölçümü olan iki histogram oluşturacak.

Koşturmak ve Analiz: bragg.mac, draw_bragg.C

makro

```
# bragg.mac
/run/initialize
/gun/particle proton
/gun/energy 150 MeV
/analysis/setFileName bragg
/run/printProgress 100
/run/beamOn 1000
```

koşu

```
source /opt/geant4/bin/geant4.sh
cp -r G4Ders G4Ders-bragg
cd G4Ders-bragg
mkdir build
cd build
cmake ../.
make -j4
./ders mip_si.mac
```

draw_bragg.C

```
void draw_bragg()
{
  TFile* f = TFile::Open("bragg.root");
  TH1* h = (TH1*)f->Get("depth");

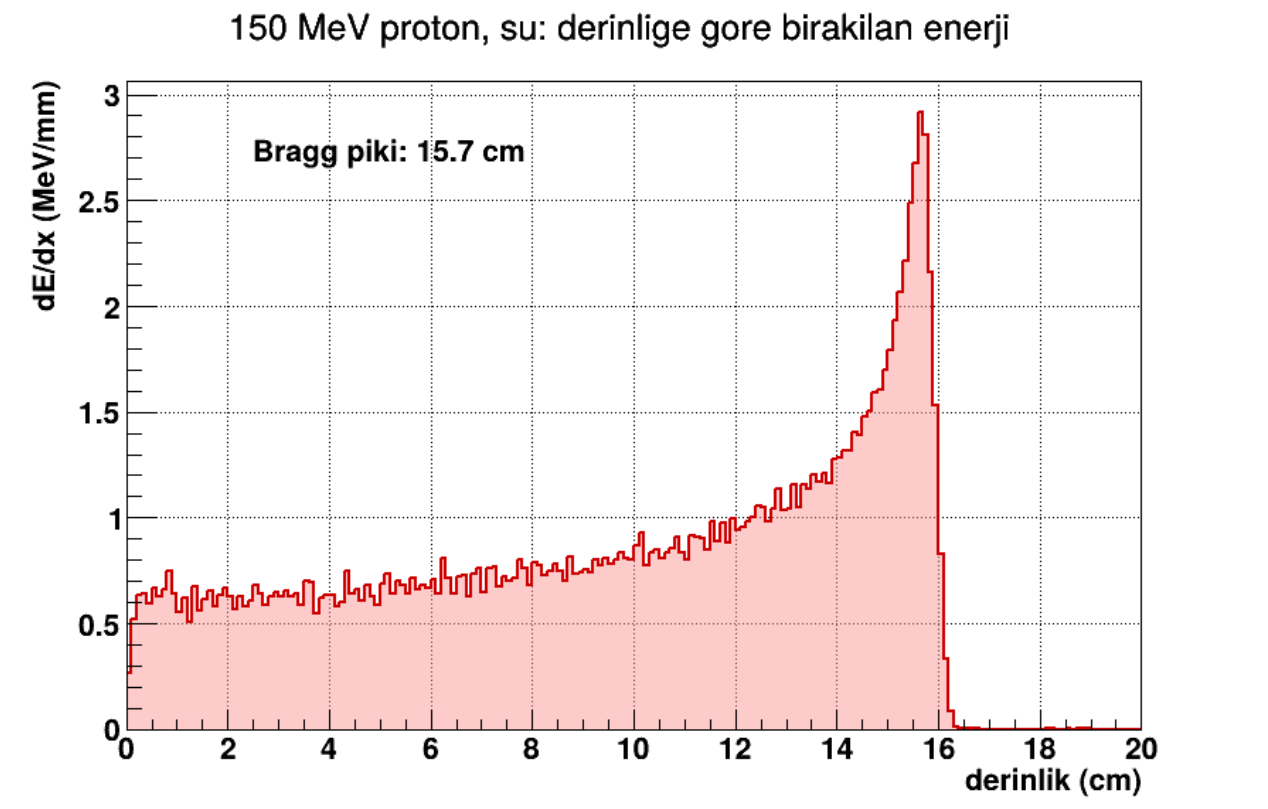
  h->Scale(1.0 / 1000);   // proton basina; kutu 1 mm -> MeV/mm

  TCanvas* c = new TCanvas("c", "", 900, 600);
  h->SetTitle("150 MeV proton, su;derinlik (cm);dE/dx (MeV/mm)");
  h->SetLineColor(kRed + 1);
  h->Draw("hist");

  double peak = h->GetBinCenter(h->GetMaximumBin());   // pikin yeri
  printf("Bragg piki: %.1f cm\n", peak);
  c->SaveAs("bragg.png");
}
```

1000 olay yeter: Her proton ≈ 360 adım atar; histogramda 360 000 giriş.

Sonuç: Bragg Tepesi



- **Giriş ≈ 0,6 MeV/mm, tepe ≈ 2,9 MeV/mm:** Proton yavaşladıkça dE/dx artar ($1/v^2$) ve durmadan hemen önce en çok.
- **Pik 15,7 cm:** NIST menzili 15,8 cm (150 MeV, su). Sonrası sıfır: proton durdu.
- **Landau ile farkı:** Orada olay başına toplamı saydık; burada her adımı derinliğine, enerjisi kadar yazdık.
- **Deneyin:** /gun/energy 100 MeV → tepe 7,7 cm; 200 MeV → menzil 26 cm, blok yetmez. /gun/particle e- → tepe yok.

Aynı iskelet, dört küçük değişiklik: Landau'dan Bragg'a. Sırada: enerji hangi sürece gidiyor? İkincil parçacıklar ve süreç adları.

Enerji hangi sürece gidiyor?

1 **PrimaryGeneratorAction**

Her olayda E0 rastgele: log10(E0/MeV) düzgün, −1 … 7 (0,1 MeV … 10 TeV)

2 **SteppingAction**

Yalnızca birincil (iz 1): sürekli kayıp + bu adımda doğan ikincillerin süreci ve enerjisi hesaba katılacak

3 **EventAction**

AddLoss(süreç, E0, enerji): if ile iyonlaşma / frenleme / çift → FillH1(2|3|4, log10 E0, enerji)

4 **RunAction**

Üç histogram: ioni, brem, pair; 16 kutu, −1 … 7 (yarım onluk)

Değişmeyen: Geometri (20 cm su), PhysicsList, main.cc.
Üç makro: /gun/particle e-, mu-, proton; 4000 olay.

- **Soru:** Enerjili parçacık suda enerjisini hangi süreçle kaybediyor?
- **Ölçüm fikri:** ikincil parçacık doğduğunda Geant4 onu oluşturan fizik sürecin adını saklar. Süreç adına GetCreatorProcess() ile erişilir. Enerjisini o sürece ait histograma yazarız.
- **Sürekli kayıp:** Kesimin altındaki küçük aktarımlar ikincil üretmez ve edep olarak gelir. O da iyonlaşmadır.

Değişiklik 1: Kaynakta Rastgele Enerji Üretimi

```
#include "Randomize.hh"   // G4UniformRand

void PrimaryGeneratorAction::GeneratePrimaries(G4Event* event)
{
  // log10(E/MeV) -1 ile 7 arasında düzgün dağılsın: her onluğa eşit olay
  G4double logE = -1. + 8. * G4UniformRand();      // 0..1 -> -1..7
  G4double energy = std::pow(10., logE) * MeV;     // 0.1 MeV .. 10 TeV
  fGun->SetParticleEnergy(energy);

  fGun->GeneratePrimaryVertex(event);   // parçacığı olaya ekle
}
```

```
// kurucudan SetParticleEnergy(1 * GeV) silinir; enerji artık her olayda
seçilir
```

- **G4UniformRand():** 0 ile 1 arasında düzgün rastgele sayı; Geant4'ün üreteci.
- **Neden logaritma?** E'yi düzgün seçsek olayların %90'ı 1 TeV üstünde olurdu. log E düzgün: her onluğa eşit olay.
- **pow(10, logE):** logaritmadan enerjiye; birim sonra çarpılır (MeV).
- **Her olayda:** GeneratePrimaries olay başına çağrılır; kurucu yalnızca bir kez. Rastgele şey buraya.
- **Makro:** /gun/energy artık etkisiz; parçacık türü /gun/particle ile.

Kontrol noktası: Derle ve çalıştır → her olay farklı enerjide fakat süreç ölçümü yok.

Değişiklik 2: SteppingAction, İkincillerin Hesabı

```
  G4Track* track = step->GetTrack();   // (hacim adı, edep, derinlik: öncekiyle aynı)
  // --- süreçler: yalnızca birincil parçacık (iz numarası 1)
  if (track->GetTrackID() != 1) return;
  G4double e0 = track->GetVertexKineticEnergy();   // başlangıç enerjisi
  // 1) adım boyunca sürekli kayıp (kesimin altı): iyonlaşma
  fEvent->AddLoss("Ioni", e0, edep);
  // 2) bu adımda doğan ikinciller: enerjiyi hangi süreç götürdü?
  const std::vector<const G4Track*>* secondaries = step->GetSecondaryInCurrentStep();
  for (size_t i = 0; i < secondaries->size(); i++) {
    const G4Track* sec = (*secondaries)[i];
    G4String process = sec->GetCreatorProcess()->GetProcessName();
    G4double energy = sec->GetKineticEnergy();
    fEvent->AddLoss(process, e0, energy);
  }
}
```

```
void AddLoss(G4String process, G4double e0, G4double energy);   // EventAction.hh
```

- **İz 1:** Birincil parçacık. İkincillerin adımlarını sayarsak aynı enerji iki kez sayılır.
- **GetVertexKineticEnergy():** İİin doğduğu andaki enerji = E0; histogramın x'i.
- **İki kayıp:** Sürekli (edep, kesim altı) ve ikincile verilen (δ, foton, çift).
- **GetSecondaryInCurrentStep():** bu adımda doğanların listesi; işaretçi, önce * ile açılır.
- **GetCreatorProcess():** ikincili yaratan süreç; adı: eIoni, eBrem, muPairProd...
- **Başlıklar:** G4Track.hh ve G4VProcess.hh eklenir.

Kontrol noktası: Henüz derlenmez: AddLoss gövdesi yok. Değişiklik 3 ve 4'ten sonra derleyin.

Değişiklik 3 ve 4: If durumu ile sınıflama

```
// EventAction.cc: süreç adına göre üç histogramdan birine
void EventAction::AddLoss(G4String process, G4double e0, G4double energy)
{
  G4double x = std::log10(e0 / MeV);
  G4AnalysisManager* am = G4AnalysisManager::Instance();
  if (process == "Ioni" || process == "eIoni" ||
      process == "muIoni" || process == "hIoni") {
    am->FillH1(2, x, energy);   // iyonlaşma (sürekli kayıp + delta elektronları)
  }
  else if (process == "eBrem" || process == "muBrems" || process == "hBrems") {
    am->FillH1(3, x, energy);   // frenleme ışıması
  }
  else if (process == "muPairProd" || process == "hPairProd") {
    am->FillH1(4, x, energy);   // çift oluşumu
  }
}
```

```
// RunAction.cc, kurucu: histogram 2, 3, 4; x = log10(E0/MeV), -1 ... 7
  am->CreateH1("ioni", "iyonlasma",        16, -1., 7.);
  am->CreateH1("brem", "frenleme isimasi", 16, -1., 7.);
  am->CreateH1("pair", "cift olusumu",     16, -1., 7.);
```

- **Süreç adları:** parçacığa göre: eIoni / muIoni / hIoni; eBrem / muBrems / hBrems; muPairProd / hPairProd.
- **Listede yoksa:** CoulombScat, Decay, hadronik... hiçbir yere yazılmaz. Bugün üç süreç.
- **x = log10 E0:** Boyutsuz bu yüzden birim etiketi olmaz. 16 kutu = yarım onluk.
- **Ağırlık = enerji:** kutu içeriği = o sürecin götürdüğü toplam enerji.
- **Kimlikler 2, 3, 4:** edep 0 ve depth 1'den sonra tanımlandılar.

Kontrol noktası: Derle; ./ders surec_e.mac, surec_mu.mac, surec_p.mac → üç ROOT dosyası (≈ 5 s her biri).

Analiz Kodu

makro (× 3: e-, mu-, proton)

```
# surec_mu.mac
/run/initialize
/gun/particle mu-
/analysis/setFileName surec_mu
/run/printProgress 1000
/run/beamOn 4000
```

koşu

```
source /opt/geant4/bin/geant4.sh
cp -r G4Ders G4Ders-bragg
cd G4Ders-bragg
mkdir build
cd build
cmake ../.; make -j4
./ders surec_e.mac
./ders surec_mu.mac
./ders surec_p.mac
root -l draw_surec.C
```

draw_surec.C

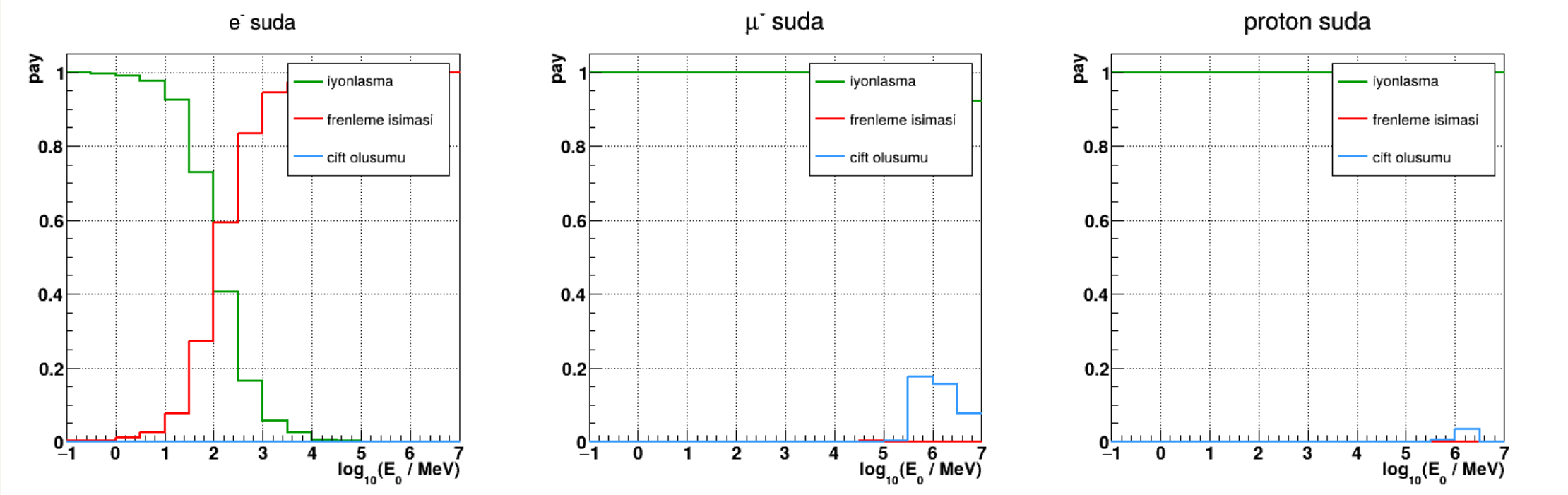
```
void panel(const char* file, const char* title)
{
  TFile* f = TFile::Open(file);
  TH1* ioni = (TH1*)f->Get("ioni");
  TH1* brem = (TH1*)f->Get("brem");
  TH1* pair = (TH1*)f->Get("pair");

  // toplam = üçünün toplamı; pay = süreç / toplam (kutu kutu)
  TH1* total = (TH1*)ioni->Clone("total");
  total->Add(brem);  total->Add(pair);
  ioni->Divide(total);  brem->Divide(total);  pair->Divide(total);

  ioni->SetTitle(Form("%s;log10(E0 / MeV);pay", title));
  ioni->Draw("hist");  brem->Draw("hist same");  pair->Draw("hist same");
}

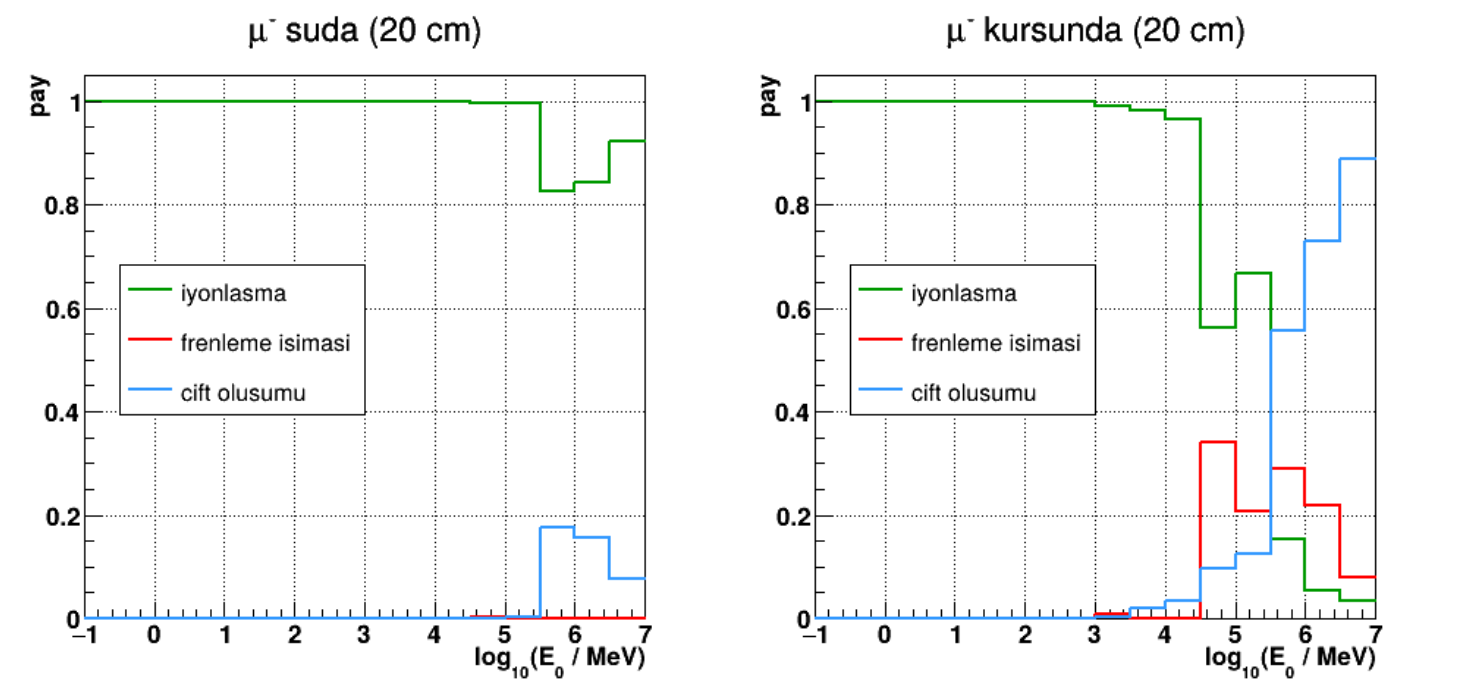
void draw_surec()
{
  gStyle->SetOptStat(0);
  TCanvas* c = new TCanvas("c", "", 1500, 500);
  c->Divide(3, 1);
  c->cd(1); gPad->SetGrid(); panel("surec_e.root",  "e^{-} suda");
  c->cd(2); gPad->SetGrid(); panel("surec_mu.root", "#mu^{-} suda");
  c->cd(3); gPad->SetGrid(); panel("surec_p.root",  "proton suda");
  c->SaveAs("surec.png");
}
```

Sonuç: Enerji kaybında Süreçlerin Payı



- **e⁻:** ≈ 100 MeV'de iyonlaşma ve frenleme eşit: kritik enerji (suda ≈ 80–100 MeV). 1 GeV üstü tamamen ışıma. Proton: her yerde iyonlaşma.
- **μ⁻:** 1 TeV üstünde çift oluşumu görünür ama tablonun altında: suda 10 TeV için λ(çift) ≈ 3 m, λ(frenleme) ≈ 300 m; 20 cm'de seyrek. Ağır hedefte görünür: sonraki slayt.

Deneyelim: Aynı kod farklı malzeme G4_Pb



- **Tek değişiklik:** FindOrBuildMaterial("G4_Pb"). Kod, makro, histogramlar aynı.
- **Kurşunda 10 TeV μ⁻:** Çift 0,89, frenleme 0,08, iyonlaşma 0,03. Suda 20 cm: çift 0,08.
- **Neden:** Işımalı tesir kesiti ∝ Z^2; yoğunluk 11×. λ(çift) ≈ 3 m → ≈ 6 cm. Kritik enerji: su ≈ 1 TeV, Pb ≈ 140 GeV.
- **Dikkat:** e^- ve proton için TeV sağanağı 20 cm kurşunda tamamen gelişir; olay başına dakikalar. Müonla deneyin.

Ders: Monte Carlo ortalamayı ancak olay yeterince sık olursa görür. Kısaca, seyrek büyük kayıplar için kalın ya da ağır hedef, ya da çok olay gerekir.

GEANT4 İLE KODLAMA · İPUÇLARI

Sık yapılan hatalar

Birimsiz sayı — Birimler

G4Box("W", 10, 10, 10) → 10 mm sanılır. Her zaman 10*cm yazın.

Tam boy vermek — Geometri

G4Box yarı boyut ister: 300 µm kalınlık için 150*um.

GeneratePrimaryVertex yok — Kaynak

Olay boş geçer, hata vermez; histogram boş kalır.

Komut sırası — Makro

/gun/... komutları /run/initialize'dan sonra.

Histogram sınırı birimsiz — Analiz

CreateH1(..., 0., 500., "keV") → 0–500 MeV! Sınırlar iç birimde: 500.*keV.

CloseFile unutuldu — Analiz

ROOT dosyası boş ya da bozuk. EndOfRunAction'da Write + CloseFile.

Hacim adı yanlış — Stepping

"silicon" ≠ "Silicon": hiçbir adım sayılmaz, histogram boş. Ad Construct'taki ile aynı olmalı.

İmza yanlış, override yok — C++

UserSteppingAction(G4Step*) const'suz → Geant4 hiç çağırmaz. override hata verdirir.

KAPANIŞ

Özet

1 · DetectorConstruction `Construct()`	Boşluk dolu dünya + 300 µm Si; solid, mantıksal, fiziksel hacim; NIST malzeme
2 · PhysicsList `RegisterPhysics`	EM option4 + bozunma + hadronik; üretim kesimi
3 · PrimaryGeneratorAction `GeneratePrimaries()`	G4ParticleGun: μ^- 1 GeV, +z, levhanın önünden; GeneratePrimaryVertex
4 · SteppingAction `UserSteppingAction()`	Levhada mı? → fEvent->AddEdep(edep)
5 · EventAction `Begin/EndOfEventAction()`	fEdep = 0 → FillH1(0, fEdep)
6 · RunAction `Begin/EndOfRunAction()`	CreateH1("edep", ..., "keV"); OpenFile, Write, CloseFile
7 · Bragg eğrisi `4 dosya, +12 satır`	Su 20 cm, proton 150 MeV; FillH1(1, z, edep) → dE/dx(z); pik 15,7 cm
8 · Süreçlerin payı `4 dosya, +25 satır`	Rastgele E0; ikincillerin GetCreatorProcess(); if ile 3 histogram; pay = süreç / toplam

Kurallar: Adımda ölç, Olayda topla, Koşuda yaz. **Sonuçlar:** Landau Dağılımı, Bragg piki 15,7 cm, e^- suda kritik enerji ≈ 100 MeV.